KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 3
상속과
클래스 변수

틀에 새긴 값과
물려받는 틀

```
class Bread:
    shop = "붕어빵 1호점"

class CreamBread(Bread):
    def __init__(self, f, p, cream):
        super().__init__(f, p)
        self.cream = cream
```

오늘의 순서

1 왜 필요한가

복사해서 만든 슈크림 틀, 틀에 새긴 값과 물려받는 틀

4–5쪽

2 클래스 변수

틀에 적은 값, 한 번에 바꾸기, 개수 세기, 두 변수 비교

6–9쪽

3 상속

물려받기, 더하기, super().__init__, 덮어쓰기, 섞어 담기

10–16쪽

4 복사와 상속 비교

고칠 곳 한 군데, 한 종류일 때만

17–18쪽

5 오류와 함정

super 빠짐, self 두 번, 재료 수, a.shop, self.count

19–23쪽

6 실습

도장 카드와 VIP 카드, 붕어빵과 미니 붕어빵 메뉴판

24–27쪽

3쪽은 노트북 준비, 28–29쪽은 막힐 때와 한 장 요약이에요.

들어가기

25에서 이어서 해요

- ✓ class, __init__, self — 24
- ✓ 메서드, a.intro() — 24
- ✓ __str__, __repr__ — 25
- ✓ 지역 변수와 전역 변수 — 20
- ✓ for, 컴프리헨션 — 13, 23

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

1. 왜 필요한가

슈크림 붕어빵 틀을 새로 만들면 거의 다 복사예요

원래 붕어빵 틀

```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        self.shop = "붕어빵 1호점"

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")
```

다른 건 cream 한 줄뿐이에요

나머지는 그대로 복사했어요. intro를 고치면 두 군데를 다 고쳐야 해요.

새로 만든 슈크림 붕어빵 틀

```
class CreamBread:
    def __init__(self, flavor, price, cream):
        self.flavor = flavor
        self.price = price
        self.shop = "붕어빵 1호점"
        self.cream = cream

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

c = CreamBread("슈크림", 1200, "많이")
c.intro()
```

```
슈크림 붕어빵 1200 원
```

가게 이름도 붕어빵마다 따로 들고 있어요

1. 왜 필요한가

틀에 새기는 값, 그리고 틀을 물려받는 틀

틀 class Bread

```
shop = "붕어빵 1호점"  ← 틀에 새긴 값
__init__(flavor, price)
intro()
```

물려받기

틀 class CreamBread(Bread)

```
cream  ← 더한 것만
나머지는 물려받아요
```

a Bread 붕어빵

flavor	"팥"
price	1000

b Bread 붕어빵

flavor	"피자"
price	1500

a와 b는 shop을 틀에서 같이 봐요

c CreamBread

flavor	"슈크림"
price	1200
cream	"많이"

c도 붕어빵

물려받은 틀로 찍어서 intro도, shop도 있어요.

클래스 변수 틀에 새겨 모두가 같이 보는 값

상속 틀을 물려받아 다른 점만 적기

2. 클래스 변수

틀에 바로 적은 값은 모든 붕어빵이 같이 봐요

[1]

```python
class Bread:
    shop = "붕어빵 1호점"                    self 없이

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.shop)
print(b.shop)
print(Bread.shop)
```

Python

```
붕어빵 1호점
붕어빵 1호점
붕어빵 1호점
```



a에 없으면 틀에서 찾아요

a에는 shop 칸이 없어요. a.shop을 꺼내면 파이썬이 a의 틀 Bread에서 찾아요.

2. 클래스 변수

틀의 값을 바꾸면 모든 붕어빵에 한 번에 바뀌어요

[2]
```python
Bread.shop = "붕어빵 2호점"
print(a.shop)
print(b.shop)
```

```
붕어빵 2호점
붕어빵 2호점
```

틀 class Bread

shop = "붕어빵 2호점"

한 줄로 모두

붕어빵이 100개여도 Bread.shop 한 줄만 바꾸면 돼요. 붕어빵마다 들고 있었다면 100번 바꿔야 해요.

바꿀 땐 틀 이름으로

Bread.shop = 처럼 틀 이름으로 바꿔요. a.shop = 으로 바꾸면 다르게 돼요(5파트).

2. 클래스 변수

찍을 때마다 틀에 개수를 세요

[3]

```python
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        Bread.count = Bread.count + 1

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
c = Bread("피자", 1500)
print(Bread.count)
```

틀의 값 +1

Python

```
3
```

찍은 줄	Bread.count
처음	0
Bread("팥", ...)	1
Bread("슈크림", ...)	2
Bread("피자", ...)	3

언제 쓰나요

가게 이름, 전체 개수, 도장 목표 개수처럼 붕어빵 하나가 아니라 틀 전체에 하나만 있으면 되는 값이에요.

2. 클래스 변수

클래스 변수와 인스턴스 변수

	인스턴스 변수	클래스 변수
어디에 적나요	`__init__ 안, self.flavor = ...`	`class 바로 아래, shop = ...`
누구 것인가요	붕어빵 하나하나 (객체)	틀 하나 (클래스)
몇 개 있나요	붕어빵 수만큼	틀에 하나
꺼내기	`a.flavor`	`a.shop, Bread.shop`
바꾸기	`a.flavor = ...`	`Bread.shop = ...`
예	맛, 가격, 도장 수	가게 이름, 전체 개수

인스턴스는 객체와 같은 말이에요. 붕어빵마다 다르면 self.이름, 모두 같으면 틀에 적어요.

3. 상속

괄호에 부모 틀을 적으면 전부 물려받아요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

class CreamBread(Bread):
    pass

c = CreamBread("슈크림", 1200)
c.intro()
print(c.flavor)
```

물려받기

Python

```
슈크림 붕어빵 1200 원
슈크림
```



부모와 자식

물려주는 Bread를 부모 클래스, 물려받는 CreamBread를 자식 클래스라고 불러요.

3. 상속

자식 틀에는 더 할 일을 적어요

[2]
```python
class CreamBread(Bread):
    def keep(self):                                    자식에만
        print(self.flavor, "붕어빵은 차갑게 보관해요")

a = Bread("팥", 1000)
c = CreamBread("슈크림", 1200)
c.intro()
c.keep()
```
Python

```
슈크림 붕어빵 1200 원
슈크림 붕어빵은 차갑게 보관해요
```

[3]
```python
a.keep()
```
Python

```
AttributeError: 'Bread' object has no attribute 'keep'
```

물려받은 것 + 더한 것

c는 부모의 intro와 자식의 keep을 둘 다 써요.

부모는 자식 것을 몰라요

팥 붕어빵 a에는 keep이 없어요. 물려받는 건 부모에서 자식 쪽으로만이에요.

3. 상속

칸을 더할 땐 super().__init__으로 부모에게 맡겨요

[4]
```python
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)          # 부모가 하던 일
        self.cream = cream                       # 더한 칸

c = CreamBread("슈크림", 1200, "많이")
print(c.flavor, c.price, c.cream)
```

```
슈크림 1200 많이
```

c — CreamBread

flavor	"슈크림"
price	1200
cream	"많이"

`super()` 부모 틀을 가리켜요. 영어로 "위의"라는 뜻이에요

`.__init__(flavor, price)` 부모의 __init__에 맛과 가격을 넣어요. self는 적지 않아요

`self.cream = cream` 자식만 가진 칸은 자식이 직접 붙여요

3. 상속

CreamBread("슈크림", 1200, "많이") 한 줄에서 일어나는 일

① **CreamBread.__init__**
자식의 __init__이 불려요. self는 새 붕어빵

② **super().__init__(flavor, price)**
부모의 __init__으로 가요. 같은 self예요

③ **self.flavor, self.price**
부모가 맛과 가격 칸을 붙여요

④ **self.cream = cream**
자식으로 돌아와 크림 칸을 붙여요

⑤ **c = 다 된 붕어빵**
세 칸이 다 붙은 붕어빵이 c에 담겨요

c — CreamBread

flavor	"슈크림"
price	1200
cream	"많이"

self는 하나예요

부모의 __init__에서도 자식의 __init__에서도 같은 새 붕어빵이에요.

3. 상속

같은 이름의 메서드를 다시 적으면 자식 것이 쓰여요

[5]
```python
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

    def intro(self):                                    덮어쓰기
        print(self.flavor, "붕어빵", self.price, "원, 크림", self.cream)

a = Bread("팥", 1000)
c = CreamBread("슈크림", 1200, "많이")
a.intro()
c.intro()
```

Python

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원, 크림 많이
```

자식부터 찾아요

c.intro()는 CreamBread에 intro가 있어서 그걸 써요.

없으면 부모에서

a는 Bread로 찍었으니 원래 intro예요.

3. 상속

부모 것을 부르고 조금만 더할 수도 있어요

[6]
```python
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

    def intro(self):
        super().intro()                          # 부모의 intro
        print("  크림:", self.cream)              # 더한 줄

c = CreamBread("슈크림", 1200, "많이")
c.intro()
```

```
슈크림 붕어빵 1200 원
  크림: 많이
```

같은 문구를 두 번 안 적어요

소개 문구는 부모 intro에만 있어요. 부모를 고치면 자식도 같이 바뀌어요.

__init__과 같은 모양

super().__init__처럼 super().intro()로 부모 메서드를 불러요.

3. 상속

섞어 담아도 붕어빵마다 자기 intro를 불러요

[7]
```python
order = [Bread("팥", 1000), CreamBread("슈크림", 1200, "보통"), Bread("피자", 1500)]
for b in order:
    b.intro()
```

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
  크림: 보통
피자 붕어빵 1500 원
```

[8]
```python
print(isinstance(c, CreamBread))
print(isinstance(c, Bread))
print(isinstance(a, CreamBread))
```

```
True
True
False
```

isinstance(값, 틀)

그 틀(또는 그 틀의 자식)로 찍었는지 True, False로 알려 줘요. 슈크림 붕어빵도 붕어빵이라 Bread에 True예요.

4. 복사와 상속 비교

복사해서 만든 틀 vs 물려받은 틀

복사 (1파트)

```
class CreamBread:
    def __init__(self, flavor, price, cream):
        self.flavor = flavor
        self.price = price
        self.cream = cream

    def intro(self):
        print(self.flavor, ...)
```

상속 (3파트)

```
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

    def intro(self):
        super().intro()
        print("  크림:", self.cream)
```

	복사	상속
부모를 고치면	두 군데 다 고쳐야 해요	자식도 같이 바뀌어요
적는 것	모든 줄	다른 점만

4. 복사와 상속 비교

"~는 ~의 한 종류"일 때 상속해요

상속	슈크림 붕어빵은 붕어빵의 한 종류예요	`CreamBread(Bread)`
상속	미니 붕어빵은 붕어빵의 한 종류예요	`MiniBread(Bread)`
상속	VIP 카드는 도장 카드의 한 종류예요	`VipCard(Card)`
따로	붕어빵 노점은 붕어빵의 한 종류가 아니에요	Stall 틀을 따로 만들어요 (25)
따로	음료는 붕어빵의 한 종류가 아니에요	Drink 틀을 따로 만들어요

코드가 비슷해 보여도 한 종류가 아니면 상속하지 않아요.

5. 오류와 함정

super().__init__을 빼먹으면 부모 칸이 없어요

[1]

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

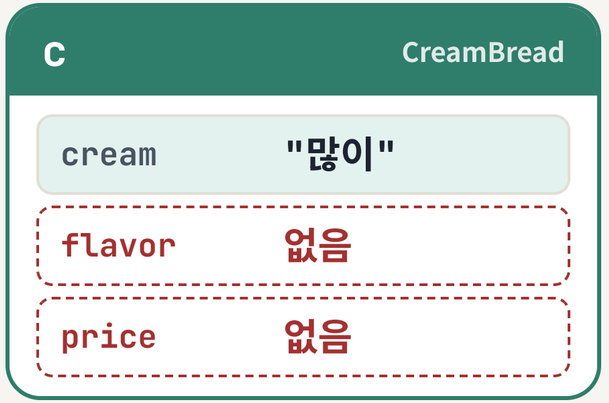
    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

class CreamBread(Bread):                                    super 없음
    def __init__(self, flavor, price, cream):
        self.cream = cream

c = CreamBread("슈크림", 1200, "많이")
c.intro()
```

Python

```
AttributeError: 'CreamBread' object has no attribute 'flavor'
```



고치기

자식 __init__ 첫 줄에 super().__init__(flavor, price)를 적어요.

5. 오류와 함정

super().__init__에 self를 넣으면 4 were given

[2]
```python
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(self, flavor, price)
        self.cream = cream

c = CreamBread("슈크림", 1200, "많이")
```
Python

self가 두 번

```
TypeError: Bread.__init__() takes 3 positional arguments but 4 were given
```

읽는 법

부모 __init__은 칸이 3개(self, flavor, price)인데 4개가 들어왔어요. self를 파이썬이 이미 넣었어요.

고치기

super().__init__(flavor, price). a.intro()에 self를 안 적는 것과 같아요.

5. 오류와 함정

자식 틀은 재료가 하나 더 필요해요

[3]
```python
class CreamBread(Bread):
    def __init__(self, flavor, price, cream):
        super().__init__(flavor, price)
        self.cream = cream

c = CreamBread("슈크림", 1200)
```
cream이 없어요 Python

```
TypeError: CreamBread.__init__() missing 1 required positional argument: 'cream'
```

읽는 법

찍을 때 불리는 건 자식의 __init__이에요. 자식은 cream까지 3개가 필요해요.

고치기

CreamBread("슈크림", 1200, "많이")처럼 자식 __init__의 칸 수에 맞춰요. 기본값(20)으로 cream="보통"을 둘 수도 있어요.

a.shop = 으로 바꾸면 a에만 새 칸이 생겨요

[4]
```python
class Bread:
    shop = "붕어빵 1호점"

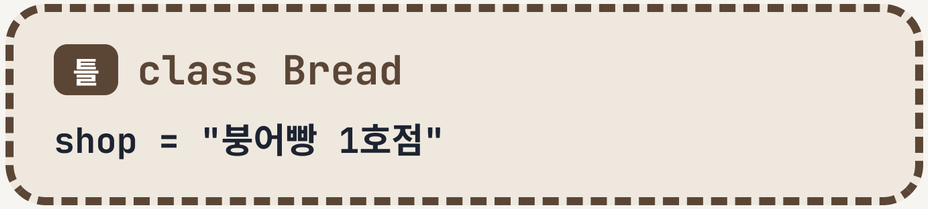
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.shop = "옆 가게"                    a에 붙이기
print(a.shop)
print(b.shop)
print(Bread.shop)
```
Python

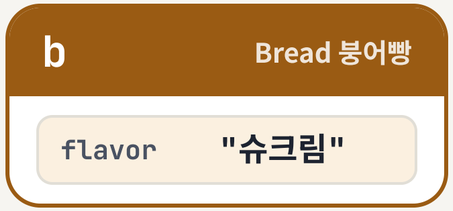
옆 가게

붕어빵 1호점

붕어빵 1호점







오류는 안 나요

24에서 손으로 칸을 붙인 것과 같아요. 틀의 값은 그대로예요. 모두 바꾸려면 Bread.shop =

5. 오류와 함정

self.count로 세면 틀의 개수가 안 늘어요

[5]

```python
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        self.count = self.count + 1

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.count, b.count)
print(Bread.count)
```

self.count

Python

```
1 1
0
```

왼쪽과 오른쪽이 달라요

오른쪽 self.count는 틀의 0을 읽어요. 왼쪽 self.count =은 붕어빵에 새 칸을 붙여요. 그래서 붕어빵마다 1이에요.

고치기

Bread.count = Bread.count + 1처럼 틀 이름으로 적어요.

6. 실습

도장 카드 ①: 틀에 가게와 목표, VIP는 두 배로

[1]

```python
class Card:
    shop = "KANT 카페"                                          # 클래스 변수
    goal = 10

    def __init__(self, name):
        self.name = name
        self.stamps = 0

    def add(self):
        self.stamps = self.stamps + 1

    def __str__(self):                                          # __str__ (25)
        left = Card.goal - self.stamps
        return f"[{Card.shop}] {self.name}: 도장 {self.stamps}개, {left}개 남음"

class VipCard(Card):                                            # add만 덮어쓰기
    def add(self):
        self.stamps = self.stamps + 2
```

Python

출력이 없는 셀이에요. 틀만 만들었어요.

6. 실습

도장 카드 ②: 같은 add인데 카드마다 다르게

[2]
```python
kim = Card("kim")
lee = VipCard("lee")
kim.add()
kim.add()
lee.add()
lee.add()
print(kim)
print(lee)
```

```
[KANT 카페] kim: 도장 2개, 8개 남음
[KANT 카페] lee: 도장 4개, 6개 남음
```

kim — Card

name	"kim"
stamps	2

lee — VipCard

name	"lee"
stamps	4

물려받은 것과 덮어쓴 것

두 카드 모두 add를 두 번 불렀어요. lee는 VipCard의 add로 4개, __str__과 가게 이름은 둘 다 Card에서 왔어요.

6. 실습

붕어빵 메뉴판 ①: 미니 붕어빵 틀을 물려받기

[3]

```python
class Bread:
    count = 0

    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        Bread.count = Bread.count + 1                    # 개수 세기

    def __str__(self):
        return f"{self.flavor} 붕어빵 {self.price:,}원"

class MiniBread(Bread):
    def __init__(self, flavor, price, qty):
        super().__init__(flavor, price)                  # 부모에게 맡기기
        self.qty = qty

    def __str__(self):                                   # __str__ 덮어쓰기
        return f"미니 {self.flavor} {self.qty}개 {self.price * self.qty:,}원"
```

Python

이 셀도 출력이 없어요.

6. 실습

붕어빵 메뉴판 ②: 섞어 담고 찍고 세기

[4]
```python
menu = [Bread("팥", 1000), MiniBread("팥", 300, 5), Bread("슈크림", 1200), MiniBread("슈크림", 400, 5)]
for m in menu:
    print(m)
print("찍은 붕어빵:", Bread.count)
print([m.flavor for m in menu if isinstance(m, MiniBread)])
```
Python

```
팥 붕어빵 1,000원
미니 팥 5개 1,500원
슈크림 붕어빵 1,200원
미니 슈크림 5개 2,000원
찍은 붕어빵: 4
['팥', '슈크림']
```

print(m)

붕어빵마다 자기 틀의 __str__ (25)

Bread.count

미니도 부모 __init__을 거쳐서 4개

isinstance

컴프리헨션(23)으로 미니만 골라요

정리

막히면 오류의 마지막 줄을 봐요

1	`AttributeError: 'CreamBread' object has no attribute 'flavor'` 자식 __init__에서 super().__init__을 안 불렀어요	`super().__init__(...)`
2	`TypeError: ... takes 3 positional arguments but 4 were given` super().__init__에 self를 넣었어요	self 빼기
3	`TypeError: CreamBread.__init__() missing ... 'cream'` 자식 틀의 재료가 모자라요	칸 수 맞추기, 기본값
4	`NameError: name 'Bred' is not defined` 괄호 안 부모 이름 오타이거나 부모를 아직 안 만들었어요	부모 셀 먼저 실행
5	**한 붕어빵만 값이 달라요** a.shop = 으로 바꿨어요	`Bread.shop =`
6	**개수가 0에서 안 늘어요** self.count로 셌어요	`Bread.count`
7	**부모 메서드를 고쳤는데 자식은 그대로예요** 자식이 같은 이름으로 덮어썼어요	`super().메서드()`

헷갈리면 "이 메서드를 자식 틀에서 먼저 찾고, 없으면 부모 틀" 순서로 따라가 봐요.

한 장 요약

shop = "1호점"

틀에 새긴 값

`Bread.shop = ...`

모두 한 번에 바꾸기

`class A(Bread):`

틀 물려받기

`super().__init__(...)`

부모에게 맡기기

`def intro(self):`

같은 이름이면 덮어쓰기

`super().intro()`

부모 것 + 조금 더

`isinstance(c, Bread)`

어떤 틀로 찍었나

~는 ~의 한 종류

상속할지 고르는 말

붕어빵마다 다르면 self.이름, 모두 같으면 틀에. 한 종류면 물려받아요

클래스 세 챕터 끝: 붕어빵 틀로 데이터와 할 일을 한 곳에 묶었어요